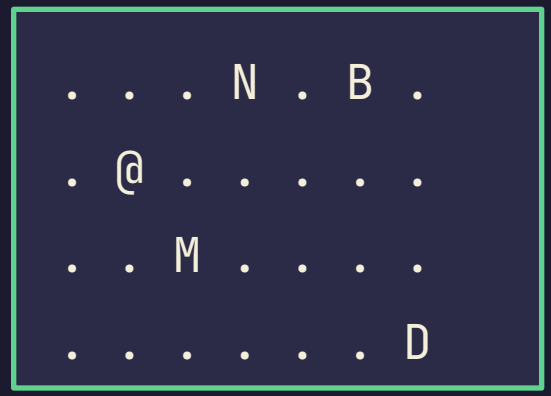
AULA DE POO · JAVA

Polimorfismo em Java

Explore o mundo de um mini-RPG e descubra como cada personagem responde do seu jeito.



Missão de hoje

Entender

O que é polimorfismo e por que ele deixa o código mais simples.

Sobrescrever

Usar herança, classe abstrata e @Override para mudar o comportamento.

Criar

Adicionar novos personagens ao jogo sem mexer no restante do código.

Ao final da aula, você cria o seu próprio personagem.

Mapa da aula

Etapa	Tempo	O que fazemos
1. Abertura	10 min	Ver o jogo funcionando
2. Conceito	20 min	Polimorfismo e classe abstrata
3. Código	20 min	Percorrer as classes do mini-RPG
4. Atividade 1	25 min	Criar o Curandeiro
5. Atividade 2	25 min	Escolher um desafio
6. Fechamento	10 min	Recapitular e discutir

Mesmo botão, resultados diferentes



NPC

Conversa com você.



Baú

Entrega ouro uma vez.



Monstro

Luta até um dos dois cair.

Em todos, você só chegou perto. Como o Java sabe o que fazer?

DEFINIÇÃO

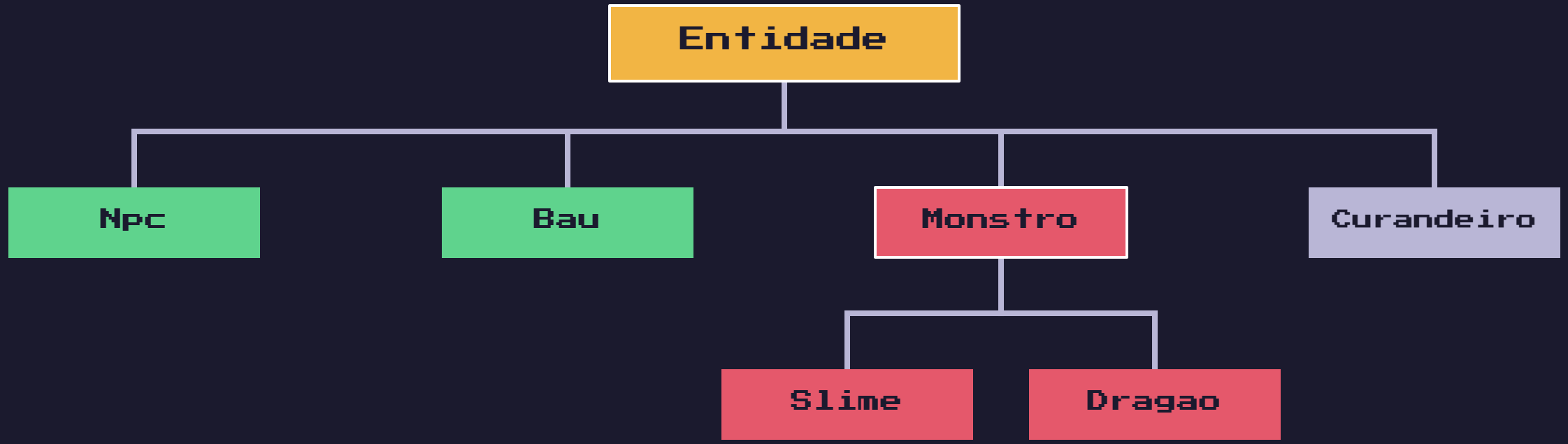
Uma chamada, muitos comportamentos

O mesmo método é chamado, mas cada objeto responde do seu jeito.

COMO NO POKÉMON

A ordem é sempre “usar ataque”. Pikachu solta um choque, Charmander solta fogo.

Quem herda de quem



A seta/linha indica **"é um"**: um **Dragao** é um **Monstro**, que é uma **Entidade**.

Classe abstrata: o contrato

```
public abstract class Entidade {
    protected String nome;
    protected int x, y;

    public abstract void
        interagir(Jogador jogador);
}
```

- abstract: não existe uma “entidade genérica” no mapa.
- interagir() não tem corpo: toda filha precisa implementar.
- Npc, Bau e Monstro cumprem o contrato do seu jeito.

Sobrescrever: cada um do seu jeito

```java
@Override
public int atacar() {
    turno++;
    if (turno % 3 == 0) {
        System.out.println("FOGO!");
        return ataque * 2;
    }
    return ataque;
}
```

Monstro.atacar()

Devolve o ataque normal.

Dragao.atacar()

Dobra o dano a cada 3 turnos.

FLEXIBILIDADE

O laço de batalha em Monstro não precisou mudar.

O momento do polimorfismo

```
Entidade alvo = new Dragao(11, 7);
alvo.interagir(jogador);
```

Se for Npc

Mostra a fala.

Se for Bau

Entrega ouro.

Se for Dragao

Inicia a batalha.

Tipo da variável: Entidade. Tipo do objeto: Dragao. É o objeto que decide.

Hora de jogar

terminal — bash

NO TERMINAL

```
$ javac *.java
$ java Jogo
```

CONTROLES

MOVIMENTAÇÃO

W A S D + Enter

SAIR DO JOGO

Pressione Q

MAPA & ENTIDADES

- `@` — Você (Jogador)
- `N` — NPC
- `B` — Baú de tesouro
- `M` — Monstro
- `D` — Dragão

MISSÃO PRINCIPAL

Visite cada tipo de entidade no jogo e anote o que ela faz.

Atividade 1: o Curandeiro

Cura grátis na primeira visita; depois, 5 de ouro por cura.

1. Abra Curandeiro.java.
2. Implemente interagir() com curar() e gastarOuro().
3. Sobrescreva simbolo() para devolver o caractere C.
4. Descomente a linha do Curandeiro em Jogo.java.
5. Compile, jogue e teste as duas visitas.

PERGUNTA-CHAVE

Você precisou mudar a lógica do Jogo?

Atividade 2: escolha um desafio

Nível	Desafio	O que fazer
Fácil	Armadilha	Causa 5 de dano ao pisar
Médio	Mercador	Vende +3 de ataque por 10 de ouro
Médio	Chefe	Estende Dragao e muda atacar()
Difícil	Interface	Crie Interagivel e faça Entidade implementá-la

Bônus: um Treinador que luta com 3 monstros em sequência.

Perguntas para pensar

1. Por que Entidade é abstrata?

2. Em Entidade e = new Dragao(...), qual interagir() roda e por quê?

3. Se criarmos 5 novos personagens, o que muda em Jogo.java?

Fase concluída!

- Herança organiza os tipos; a classe abstrata define o contrato.
- @Override muda o comportamento em cada subclasse.
- Novos tipos entram no jogo sem alterar o código existente.

Próxima missão: interfaces e novos poderes.